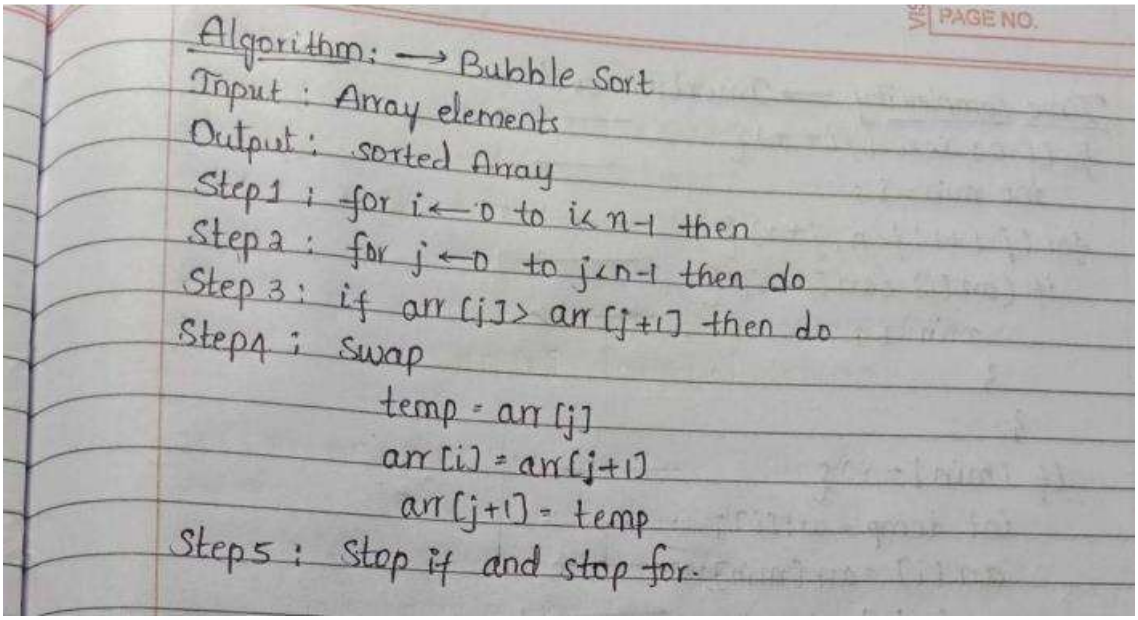
Practical –1

Aim: To Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

1)Bubble sort

Algorithm:



Program:

```
#include <iostream>

using namespace std;

int main() {

    int n;

    cout << "Enter the number of elements: ";

    cin >> n;

     int arr[n];

    cout << "Enter " << n << " elements: ";
```

Name: V.Bhavya sri
Enrollment no: 92460118227

```
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

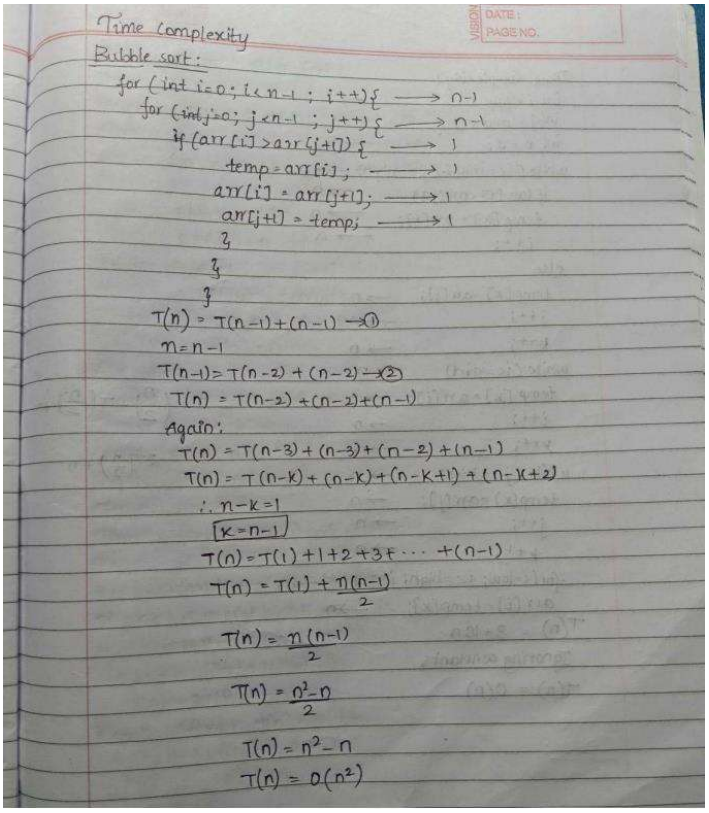
Output:

```
Enter the number of elements: 5
Enter 5 elements: 9
3
7
5
1
Sorted array: 1 3 5 7 9

=== Code Execution Successful ===
```

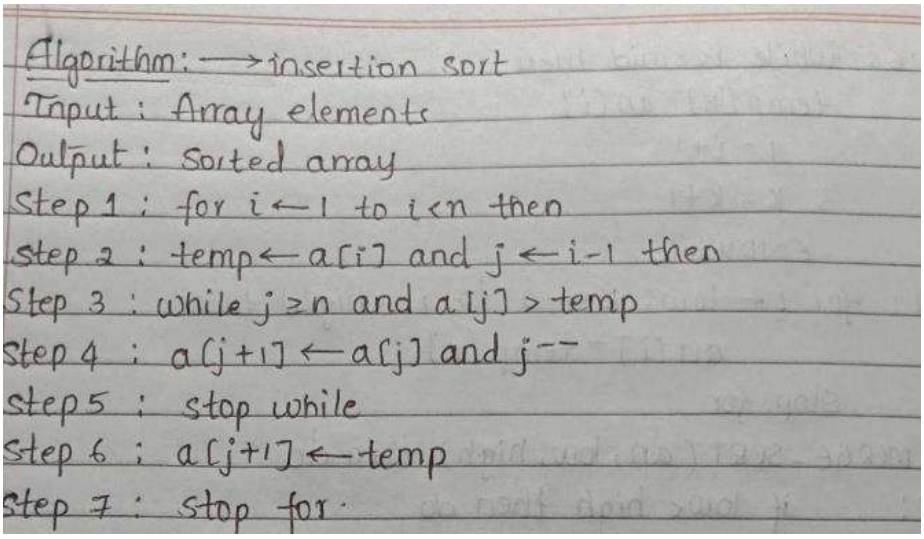
Time complexity:

Name: V.Bhavya sri
Enrollment no: 92460118227



2) Selection Sort

Algorithm:



Program:

```
#include <iostream>
```

Name: V.Bhavya sri
Enrollment no: 92460118227

```
using namespace std;
int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    for (int i = 0; i < n - 1; i++) {
        int key = arr[i];
        // Selection sort logic
    }
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

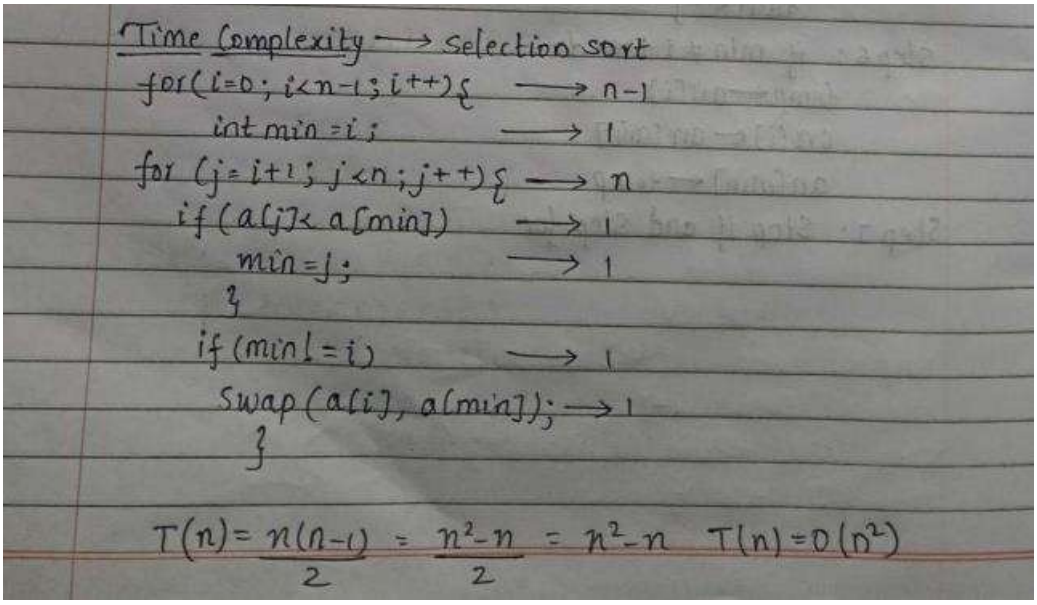
Name: V.Bhavya sri
Enrollment no: 92460118227

Output:

```
Enter the number of elements: 5
Enter 5 elements: 1
4
3
6
9
Sorted array: 1 3 4 6 9

=== Code Execution Successful ===
```

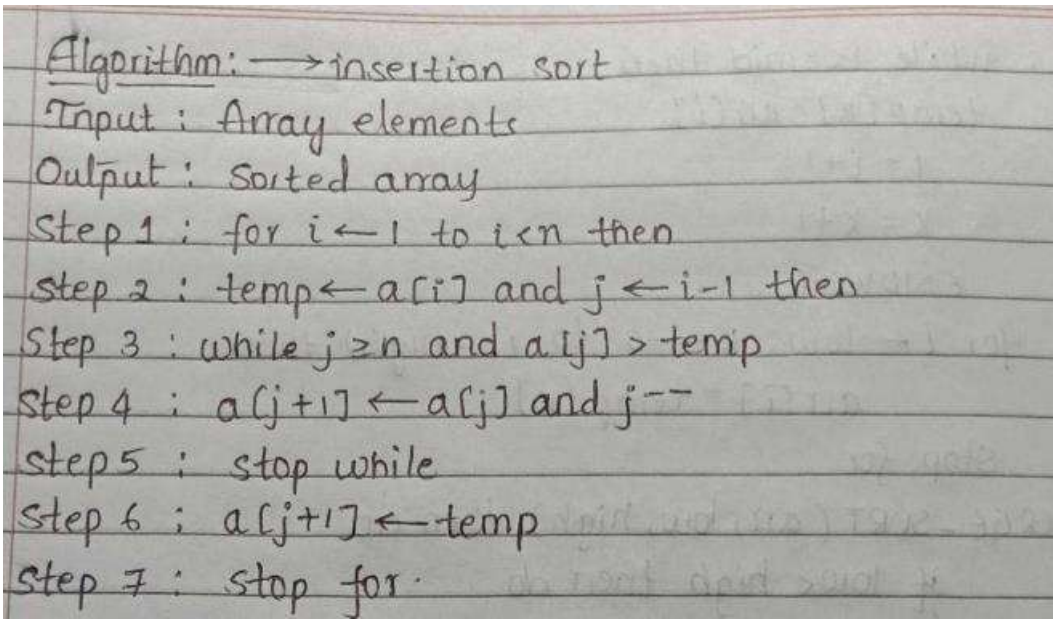
Time complexity :



Name: V.Bhavya sri
Enrollment no: 92460118227

3) Insertion sort:

Algorithm:



Program:

```
#include <iostream>

using namespace std;

int main() {

    int n;

    cout << "Enter the number of elements: ";

    cin >> n;

    int arr[n];

    cout << "Enter " << n << " elements: ";

    for (int i = 0; i < n; i++) {

        cin >> arr[i];

    }

    for (int i = 1; i < n; i++) {
```

```
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```

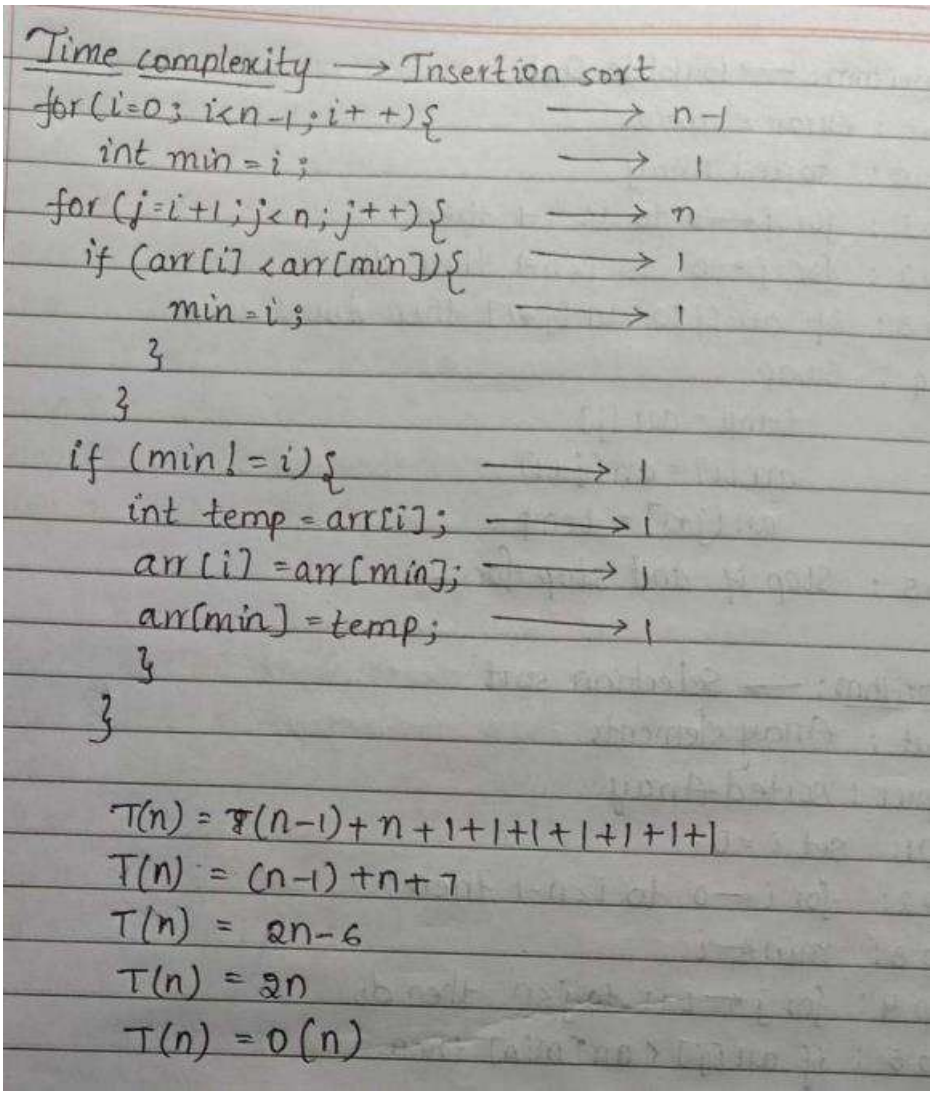
Output:

```
Enter the number of elements: 5
Enter 5 elements: 1
4
3
6
9
Sorted array: 1 3 4 6 9

=== Code Execution Successful ===
```

Name: V.Bhavya sri
Enrollment no: 92460118227

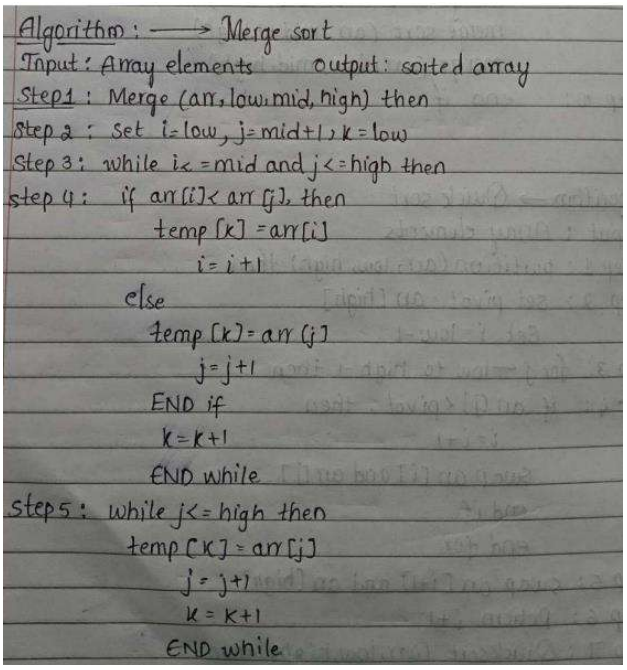
Time complexity:

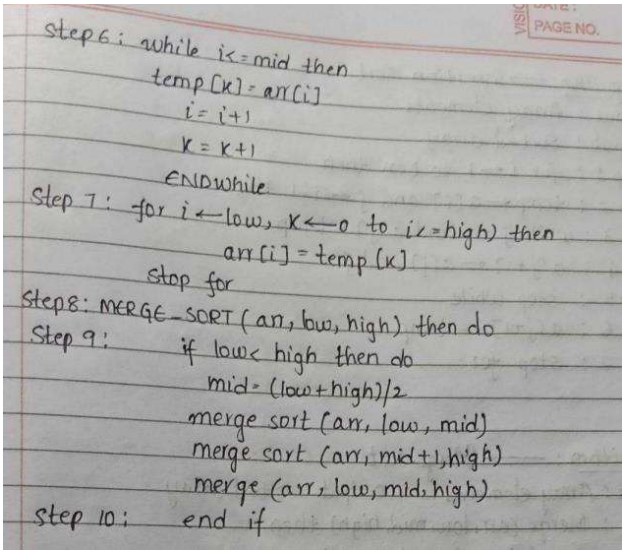


Name: V.Bhavya sri
Enrollment no: 92460118227

4) Merge sort

Algorithm:


Algorithm : ——→ Merge sort
Input : Array elements output : sorted array
Step1 : Merge (arr, low, mid, high) then
Step 2 : Set i=low, j=mid+1, k=low
Step 3 : while i<=mid and j<=high then
step 4 : if arr[i]<arr[j], then
temp[k]=arr[i]
i=i+1
else
temp[k]=arr[j]
j=j+1
END if
k=k+1
END while
step 5 : while j<=high then
temp[k]=arr[j]
j=j+1
k=k+1
END while


step 6 : while i<=mid then
temp[k]=arr[i]
i=i+1
k=k+1
ENDwhile
Step 7 : for i←low, k←0 to i<=high) then
arr[i]=temp[k]
Stop for
Step8 : MERGE-SORT (arr, low, high) then do
Step 9 : if low< high then do
mid=(low+high)/2
merge sort (arr, low, mid)
merge sort (arr, mid+1, high)
merge (arr, low, mid, high)
step 10 : end if

Program:

```
#include <iostream>

using namespace std;
```

```
void merge(int arr[], int low, int mid, int high) {
    int i = low;
    int j = mid + 1;
    int k = 0;
    int temp[high - low + 1];
    while (i <= mid && j <= high) {
        if (arr[i] < arr[j]) {
            temp[k] = arr[i];
            i++;
        }
        else {
            temp[k] = arr[j];
            j++;
        }
        k++;
    }
    while (i <= mid) {
        temp[k] = arr[i];
        i++;
        k++;
    }
    while (j <= high) {
        temp[k] = arr[j];
        j++;
        k++;
    }
```

```
    for (int i = low, k = 0; i <= high; i++, k++) {
        arr[i] = temp[k];
    }
}
void mergeSort(int arr[], int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);
        merge(arr, low, mid, high);
    }
}
int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[n];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    mergeSort(arr, 0, n - 1);
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
```

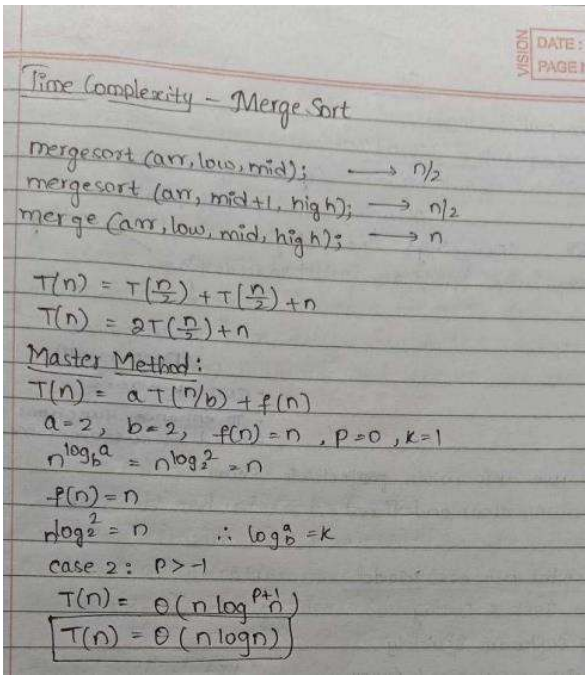
```
    }
    return 0;
}
```

Output:

```
Enter number of elements: 6
Enter 6 elements: 5
7
2
1
4
5
Sorted array: 1 2 4 5 5 7

=== Code Execution Successful ===
```

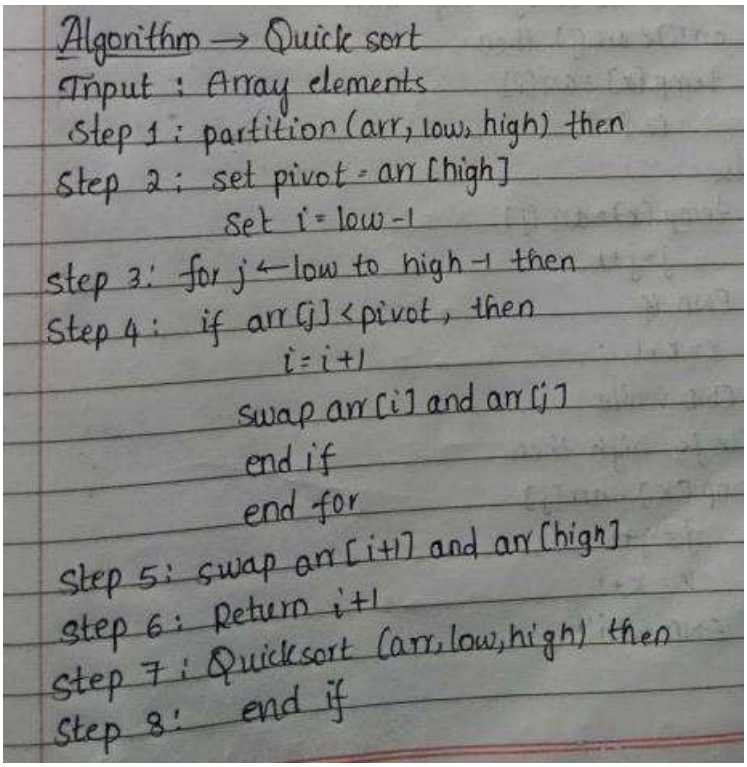
Time complexity:



Name: V.Bhavya sri
Enrollment no: 92460118227

5) Quick sort

Algorithm:



Program:

```
#include <iostream>

using namespace std;


int partition(int arr[], int low, int high) {


    int pivot = arr[high];

    int i = low - 1;


    for (int j = low; j < high; j++) {
```

```
        if (arr[j] < pivot) {

            i++;

            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

void quickSort(int arr[], int low, int high) {

    if (low < high) {

        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);

        quickSort(arr, pi + 1, high);
```

```
    }
}

int main() {

    int n;

    cout << "Enter number of elements: ";
    cin >> n;

    int arr[n];

    cout << "Enter " << n << " elements: ";

    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    quickSort(arr, 0, n - 1);

    cout << "Sorted array: ";

    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
```

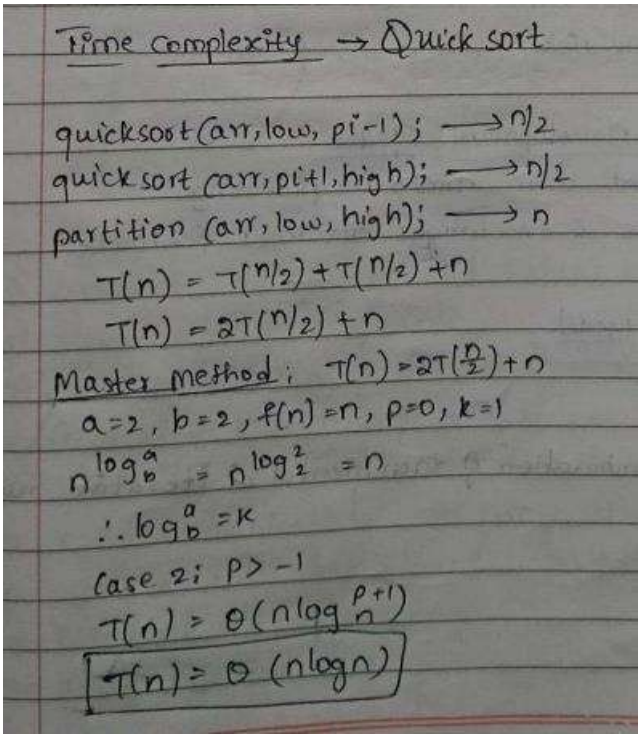
```
    return 0;
}
```

Output:

```
Enter number of elements: 5
Enter 5 elements: 9
2
6
1
6
Sorted array: 1 2 6 6 9

=== Code Execution Successful ===
```

Time complexity:



Conclusion:

The Bubble Sort algorithm successfully sorts the given elements in ascending order by repeatedly comparing adjacent elements and swapping them when they are in the wrong order. The algorithm is simple and easy to implement, but it is not efficient for large datasets. Its best-case time complexity is **O(n)** when the array is already sorted, while the average-case and worst-case time complexities are **$O(n^2)$**. Therefore, Bubble Sort is mainly suitable for small datasets and educational purposes.

Name: V.Bhavya sri
Enrollment no: 92460118227